EEP 522
Assignment3
Gesture-Control MP3 Player

Matthew Russo

Contact: mcrusso@uw.edu

Date: 3/14/2026

Contents

Abstract 3
Introduction 3
Method 3
Results 8
Discussion 8
Conclusion 10
References 11

Figures

Figure 1: Speaker Connected 4
Figure 2 Playing Music 5
Figure 3: Astronaut Elvis Speaker Connected 6
Figure 4: Gesture Sensor Wiring 7
Figure 5: Enable I2C on Raspberry Pi 7
Figure 6: RPi I2C Messages 8
Figure 7: Trust Speaker 9
Figure 8: Amixer Volume of PCM 9

Abstract

This report covers the process and hurdles for developing a wireless gesture-controlled MP3 player using a Raspberry Pi 4. The goal was to use and apply the characteristics of the Pi learned in Assignment 2 and produce a working project. During the development, it became clear that more characteristics had to be learned to get a working build. This build is to be a prototype for a gesture-controlled music player for users at home for mass-production. Through great effort, a working prototype build was successfully developed.

Introduction

The final goal of this class was to design and develop a wireless gesture-controlled MP3 player. Music players are very common as phones and PCs have a player pre-installed on them. The common trait in all of them is that they require some form of button input to control them. Even wireless ear buds have a form of touch control on them. This gesture-controlled music player will take steps towards removing that physical input from the user.

The usefulness of having gesture-controlled music control would be for at home media systems. A large audience of music listeners listen to music while studying at home. The simple action of just changing songs or lowering the volume can interrupt people while trying to focus. The simple system as it stands has the user switching tabs to the media player to make the controls. The goal of this new system is to be more efficient by removing the need for the user to have to switch tabs or look at their phone to control the music. This product's end market would be for mass production. Again, while the current system is simple, it still can have the effect of causing people to lose focus on the work they want to focus on.

This report will explain the procedure of how to setup a gesture-controlled MP3 player using a Raspberry Pi 4. It will show the best tools to use, the structure of the design, and the problems that were had during development. This design only uses a Pi, a speaker, and a sensor, but even though the parts are limited, the effort that is needed to get them to work together is complicated.

Method

The first step for a wireless music player is to connect to a speaker. I have a Bose Bluetooth (BT) speaker that I need to connect my Pi to. The Pi has a program called 'bluetoothctl' that we will use to find and connect to the speaker. The BT chip must be turned on with 'power on' command. Then the 'scan' command can be used to find any active BT device and will show the device's MAC address. When the speaker is found, I use the 'connect' command with the MAC address to establish a connection between the Pi and speaker.

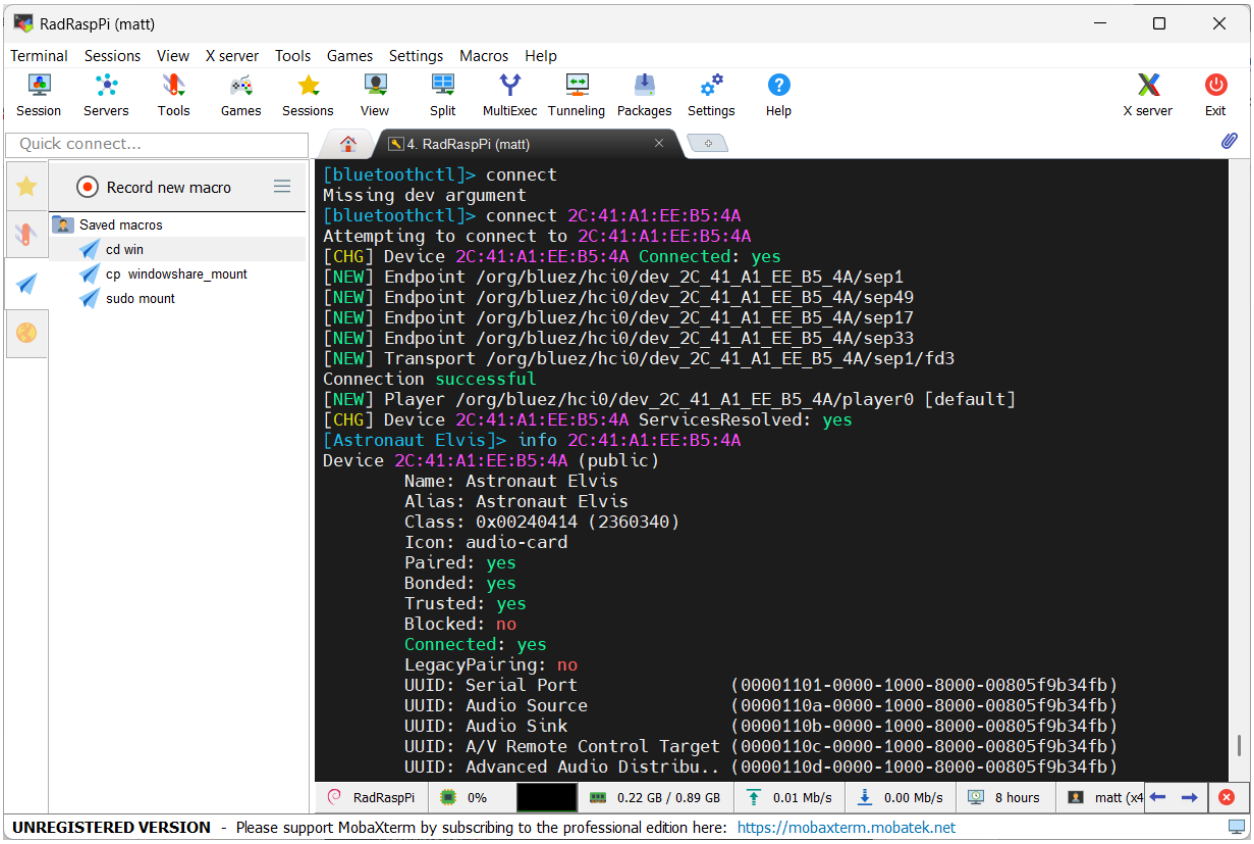


Figure 1: Speaker Connected

To test that the speaker is working, it is now time to play a song. I copied a couple of mp3 files and stored them onto the Pi in a folder marked “Playlist”. It is helpful to keep all songs located within the same folder.

There are many media player applications available for the Pi. I will be using the mpg123 as it was recommended by more people on forum pages. Mpg123 can be installed with ‘sudo apt-get install mpg123’. Change the directory to the location of the playlist. Run mpg123 with the name of a song, the music will start playing over the speaker!

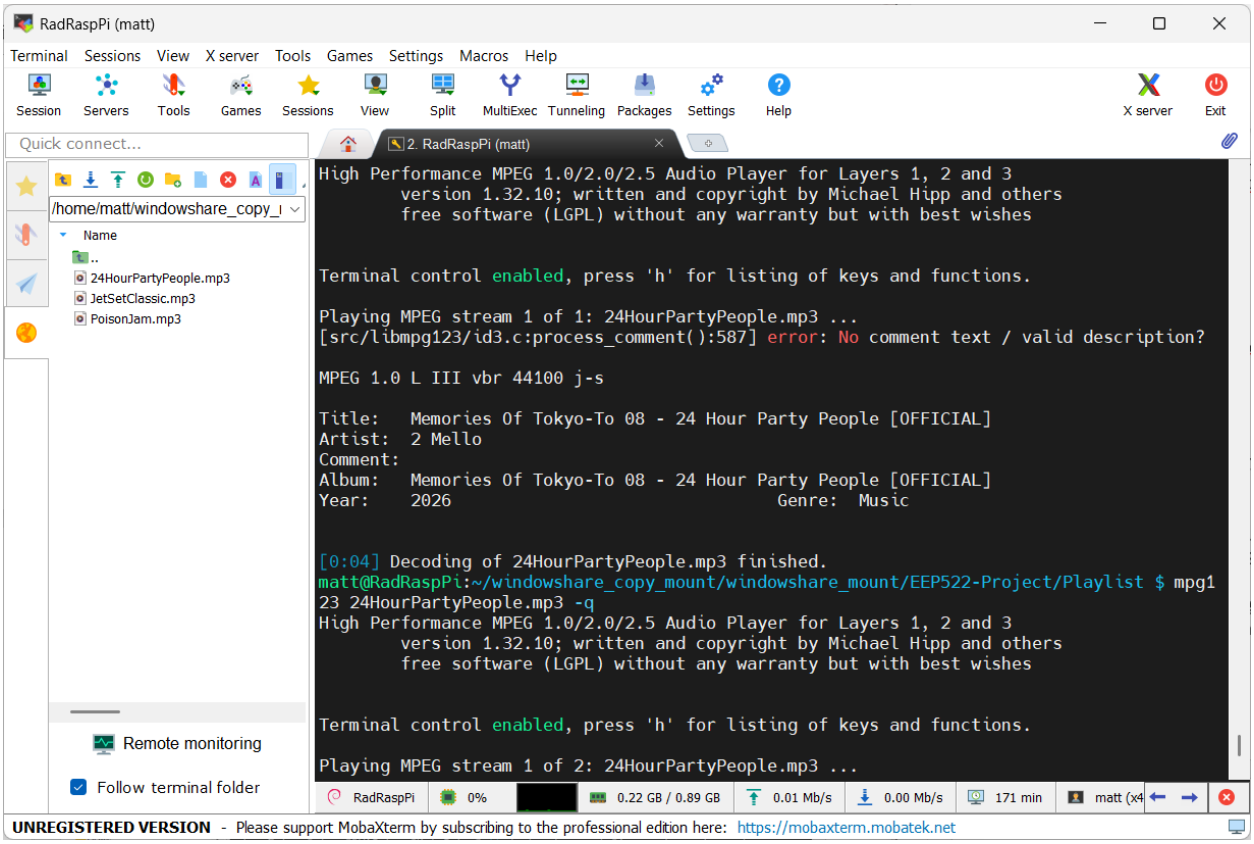


Figure 2 Playing Music

It is difficult to show audio in a written report, but the output on the terminal will look like the above figure, but with a different song title. Now if you have the same output, but no audio, it might mean the volume is too low.

Next step is to change the volume. The best way to change the volume is to use WirePlumber. Also know as pipeWire, it is a multimedia server designed to handle audio and video on linux that is pre-installed on Raspberry Pis. WirePlumber allows a user to change the volume by set amounts, percentages, or a specific volume. To change the volume, you first must learn the Sink ID of the speaker. Find your speaker in WirePlumber by running the command “wpctl status”.

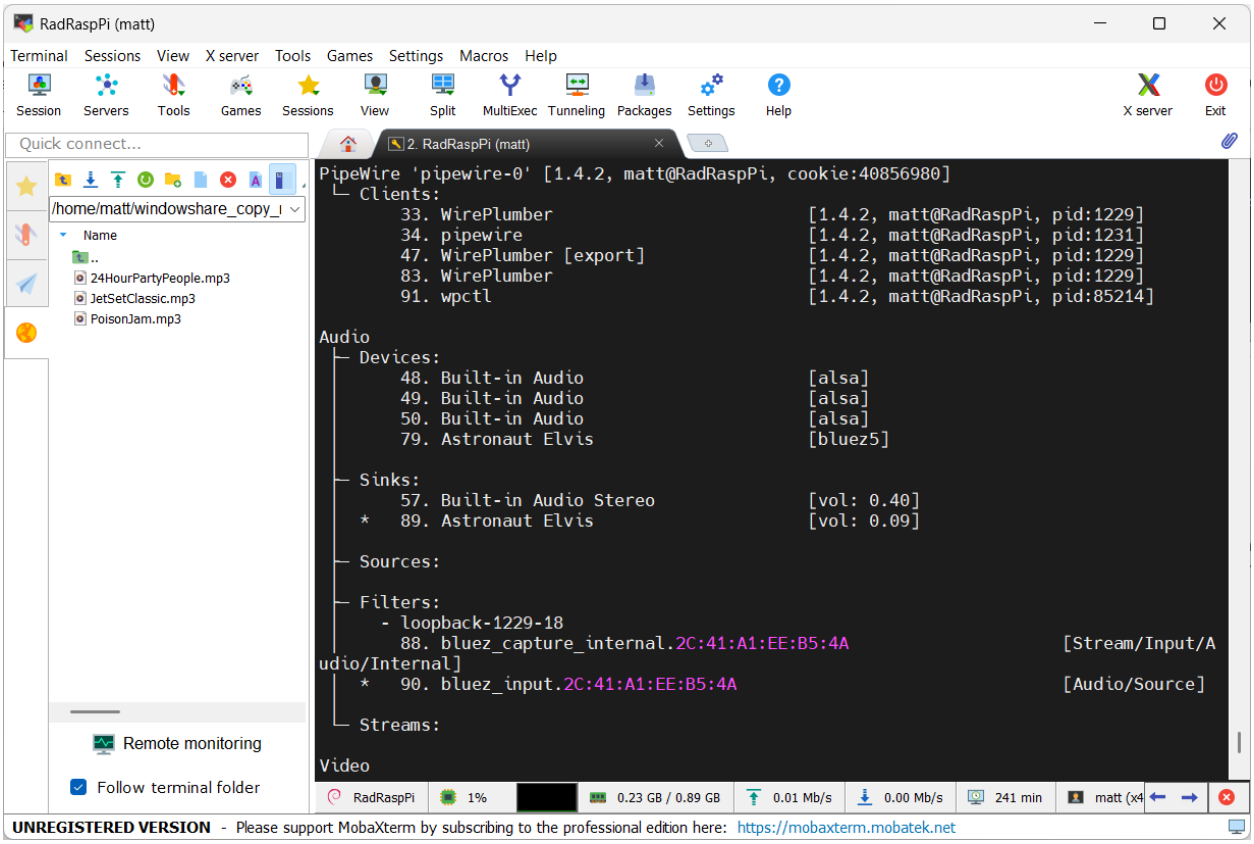


Figure 3: Astronaut Elvis Speaker Connected

The above image shows my connected BT speaker under Audio. For the volume of the device, we look at the Sinks. My speaker is called Astronaut Elvis and the volume is set to 9. To increase the volume to a louder level, we need the Sink ID of 89. Now by using "wpctl set-volume 89 0.2+" the volume will increase by 20 and become 29.

With the speaker working with the Pi, the last step in the setup is to connect the gesture sensor to the Pi. The job of the sensor is to send messages to the Pi to control the speaker. We have established a working connection with the Pi and speaker, so now is the time to make the connection with the sensor and Pi.

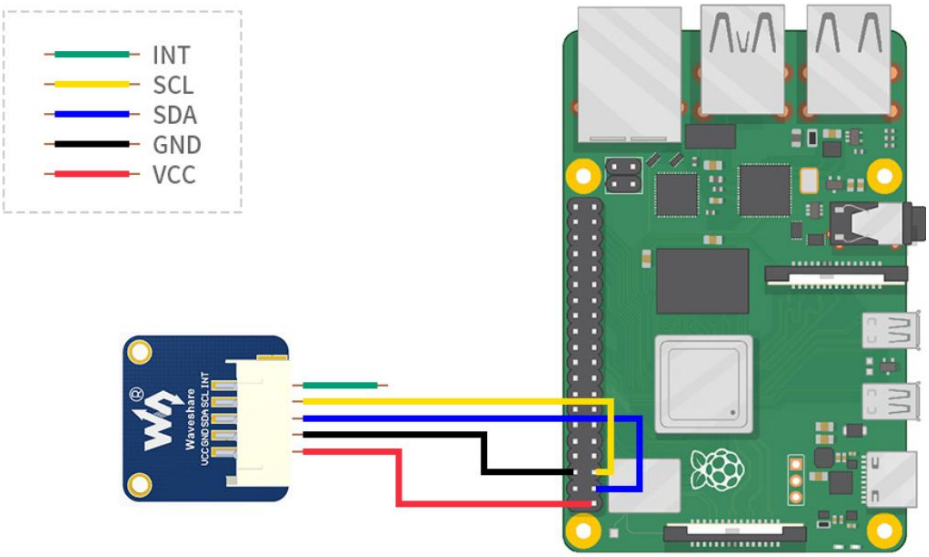


Figure 4: Gesture Sensor Wiring

The gesture sensor being used is the Grove Gesture by Seeed Studios. It uses the PAJ620U2 sensor and communicates with I2C. The above diagram is a representation of how to wire them together. I did not have a proper connector, so I used a breadboard to wire them. The I2C data and clock lines are on pins 3 and 5 respectively.

When the sensor gets power, it will start to communicate on the bus, but we still must enable I2C. Using raspi-config, check the “Interface Options” for I2C and enable it.

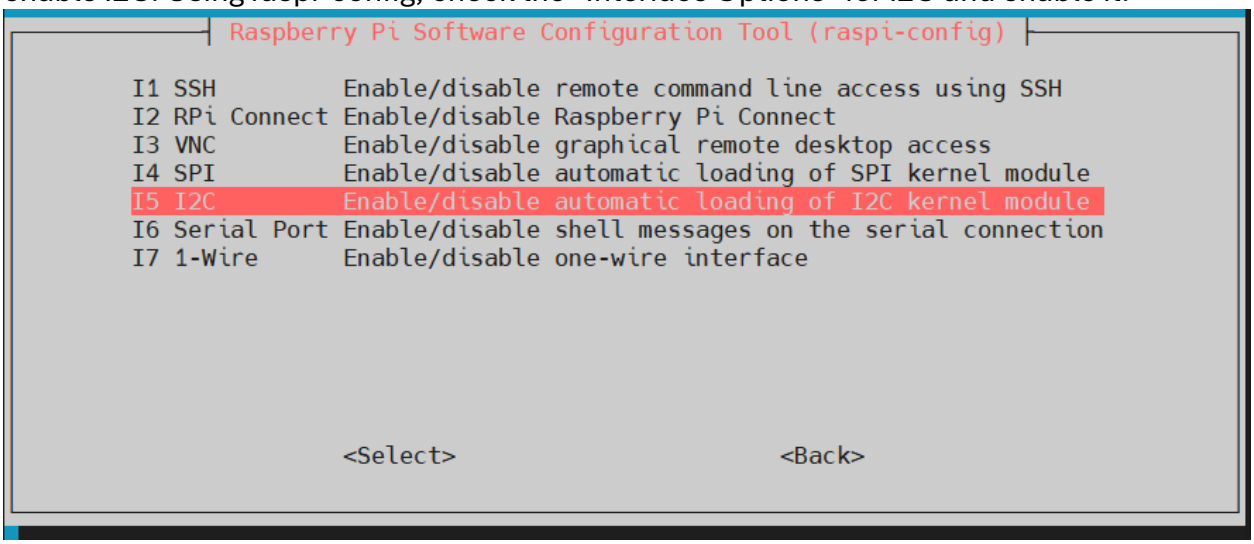
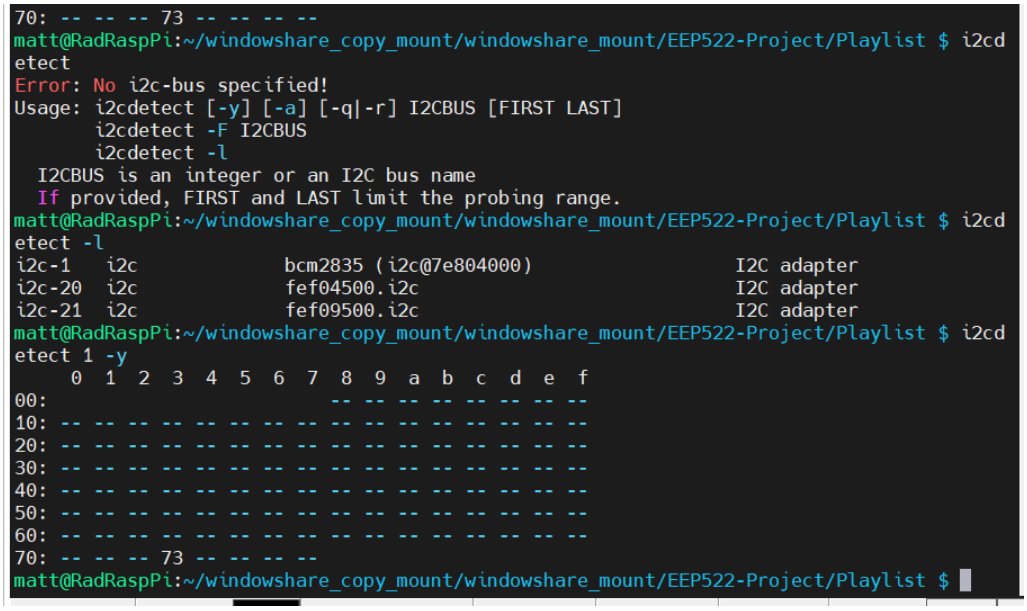


Figure 5: Enable I2C on Raspberry Pi

We must validate that the sensor is talking on the bus line. Using “I2Cdetect 1 -y” to check the ID devices on I2C bus 1 on the Pi.



```
70: -- -- -- 73 -- -- -- --
matt@RadRaspPi:~/windowshare_copy_mount/windowshare_mount/EEP522-Project/Playlist $ i2cd
etect
Error: No i2c-bus specified!
Usage: i2cdetect [-y] [-a] [-q|-r] I2CBUS [FIRST LAST]
       i2cdetect -F I2CBUS
       i2cdetect -l
  I2CBUS is an integer or an I2C bus name
  If provided, FIRST and LAST limit the probing range.
matt@RadRaspPi:~/windowshare_copy_mount/windowshare_mount/EEP522-Project/Playlist $ i2cd
etect -l
i2c-1   i2c             bcm2835 (i2c@7e804000)                  I2C adapter
i2c-20  i2c             fef04500.i2c                            I2C adapter
i2c-21  i2c             fef09500.i2c                            I2C adapter
matt@RadRaspPi:~/windowshare_copy_mount/windowshare_mount/EEP522-Project/Playlist $ i2cd
etect 1 -y
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                         -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- 73 -- -- -- --
matt@RadRaspPi:~/windowshare_copy_mount/windowshare_mount/EEP522-Project/Playlist $
```

Figure 6: RPi I2C Messages

My sensor is transmitting with ID 0x73 and is visible on the table – we have verified communication. With the sensor communicating, it will be possible to send its initialization values. The initialization is needed to get the sensor to output data from the hand motions.

With all communications ready, the gesture-controlled music player is ready to operate. Now all the user has to do is compile the code and run the program.

Results

I developed a fully working gesture-controlled music player! While there are no quantitative numbers being recorded for this system, the goal was to have a functional build, which was achieved. I can increase and decrease the volume, pause the song playing, and skip to the next song in the playlist, all without clicking a button. It will even detect when a song is finished playing and automatically plays the next song. The sound quality of the music sounds great with no distortion. Plus, with using mpg123, it is possible to add base and other features into the playing audio. Meaning the project is in a position to scale with extra features.

Discussion

A surprising number of problems were identified during this project. The first was turning on BT. All guides online have the same steps to getting BT working. But whenever I tried to power it on, I would keep getting the error “Failed to set power on: org.bluez.Error.Failed”. I could find other people in forums with the same error, but not a working solution.

The problem was that BT was blocked by another program. By default, the Raspberry Pi 4 has BT blocking. To unblock it you have to call “rfkill unblock Bluetooth”. This simple line was stopping me from getting started with the project. While this solution is visible to other people with the issue online, I overlooked it because I figured since I could use the bluetoothctl tool, that meant I was not being blocked.

A useful tip in BT to avoid having to use the ‘connect’ feature on startup every time is to ‘trust’ the speaker. If the device is trusted, the Pi will connect to it automatically on startup. It is simple quality of life step to have when prototyping the system.

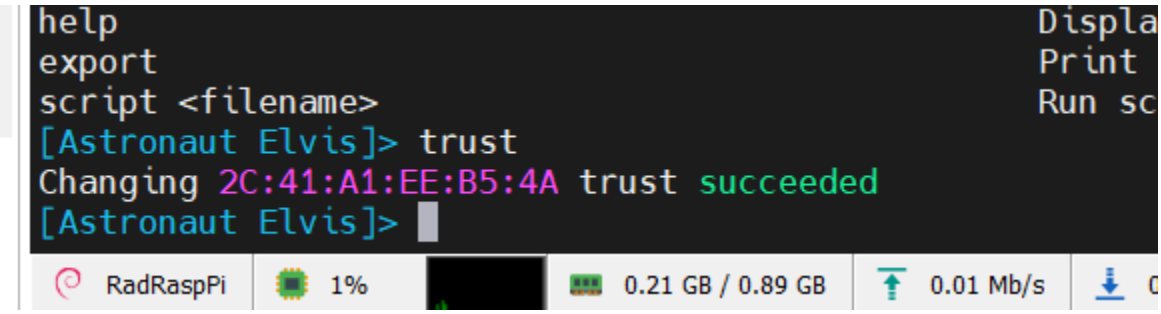


Figure 7: Trust Speaker

The next problem I endured was changing the volume. The normal way people change the volume on the Pi is using the Amixer program. That will not work. Amixer changes the volume using the PCM (Pulse Code Modulation). That is used for audio over wired interfaces. With BT being wireless, it will not work. The audio that is sent to BT comes from a different source. This is why PipeWire must be used as it will provide the correct audio control for the wireless speakers.

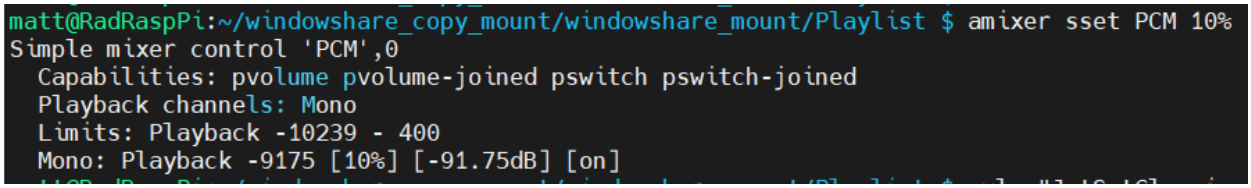


Figure 8: Amixer Volume of PCM

Another problem I had was using the Grove gesture sensor. Even though I had wired it up correctly, I was not getting any values to output from it. The documentation never told me I had to initialize the sensor. I then referenced through Seeed’s website for instruction on how to use the sensor, but solutions they had are for pre-built code for systems that required additional hardware. I continued to try to use that code and re-format it for my system, but with minimal comments, it proved time consuming with no results. Thankfully, I was able to find a library from another website that would be useful for getting my sensor initialized.

The last issue discovered (besides debugging code) was using I2C interrupt. I have previously used STMs, so I have experience with using IRQs. It was quite a surprise to find out that GPIO library I use, WiringPi, does not have an interrupt for I2C. While my original plan was to operate with my speaker with IRQs, I just had to change to a polling system for the messages.

The reason IRQs were wanted was that the main C code was supposed to focus on playing music. When a command from the sensor came in, the IRQ would allow me to change songs and volume. The main characteristics I was learning from Assignment 2 was the speed of using IRQs. The original outline for my code was to be using IRQs, but now with that no longer an option, I had to learn how to use pipes in my code. I have never had experience with using pipes, so I have learned that while they are simple to understand, it is not easy implementing. While I was able to pivot my plan to this new situation, it was still surprising that the characteristics I was working on would end up not being used in my final design.

I have to use pipes because my C code is working with the music player mpg123, the C program is not playing the mp3 files directly. This means my program must interface with another one to feed it commands and to read the output. Different methods were tried before the pipe method, such as background task '&', nohup, and system(), but each one of them had a reason for failing my situation. I can say confidently that usings pipes is the best way to handle communicating between different programs.

Conclusion

Over the course of this project, I was able to connect with BT, interface with I2C, and control multiple programs with the Raspberry Pi 4. All these parts and programs were able to be combined into a gesture-controlled music player using a BT speaker.

Now while still a prototype build, it still shows the feasibility of allowing users to maintain focus on any task while controlling the music. The best way to make this product better for market is to increase the number of features it can accomplish. Using the gesture sensor, it is still possible to randomize playlist, repeat current track, and even control base. By adding in features like these, it is reasonable to sell the product as a media control device for at home media systems.

Extra hardware can also be added to this design. I had plans to add a screen to be displaying the song playing, but ran out of time before I could attempt to implement it. I had already lost track of the time spent on just getting BT working, but my guess would be over 40 hours on this. But I can see a version 2 of this design by using a camera for 'hand detection' instead of using the gesture sensor. Showing that both simple and complex

pieces of hardware can be integrated into this design, increasing in appeal for mass production.

Having a product that does not require any visual control of their music is what people need to improve concentration in their work. My current prototype shows the gesture-controlled music play is useable, but with extra work and features, it can be sold as a full product.

References

[1] author, P. F. (2023, April 15). How to Use Bluetooth on Raspberry Pi: GUI & Command Guide. RaspberryTips. https://raspberrytips.com/raspberry-pi-bluetooth-setup/

[2] PAJ7620U2 Gesture Sensor - Waveshare Wiki. (2025). Waveshare.com. https://www.waveshare.com/wiki/PAJ7620U2_Gesture_Sensor

[3] PAJ7620U2 Product Datasheet Integrated Gesture Recognition Sensor PAJ7620U2: Integrated Gesture Recognition Sensor General Description. (n.d.). Retrieved March 15, 2026, from https://files.seeedstudio.com/wiki/Grove_Gesture_V_1.0/res/PAJ7620U2_DS_v1.5_05012022_Confidential.pdf

[4] WiringPi (Unofficial Mirror/Fork). (2022, May 2). GitHub. https://github.com/WiringPi/WiringPi